Understanding the Math Behind

microgpt by Andrej Karpathy

Partial Derivatives · Tensor Calculus · Autograd · Full Jacobians · Industry Implementations

AUTHOR

Oluwaseyi Awoga

TOPIC

Backpropagation & Matrix Calculus

Partial Derivatives

Chain Rule

Jacobian Matrix

DAG & Topo Sort

Industry Shortcut

Verify

OVERVIEW

Table of Contents

This deck builds up from scalar partial derivatives through tensor calculus to full autograd backpropagation with Jacobians, chain rule worked examples, DAG topological sorting, and all operations in MicroGPT.

Introduction to Partial Derivatives with Scalar Inputs

Scalar Inputs

- **Scalar Inputs:** Define x_1 and x_2 as real numbers ($x_1, x_2 \in \mathbb{R}$)
- **Objective:** Examine the derivative of output $y = f(x_1, x_2)$ with respect to each input independently.
- **Focus:** Derivatives for various arithmetic operations on x_1 and x_2
- **Importance:** Understand how changes in each input affect the output individually.

Partial Derivatives of Addition

Function definition: $f_+(x_1, x_2) = x_1 + x_2 = y_+$

We want $\partial/\partial x_1$ and $\partial/\partial x_2$:

```
∂/∂x1 (x1 + x2) = 1
∂/∂x2 (x1 + x2) = 1
```

- Derivatives are symmetric
- Changing x_1 or x_2 by ε results in a $1 \times \varepsilon = \varepsilon$ change in y

Key Idea:

A partial derivative isolates how ONE input affects the output while treating all other inputs as constants. This is the foundational building block of all backpropagation.

Partial Derivatives of Subtraction & Multiplication

Partial Derivatives of Subtraction

Function definition:

$$f_-(x_1, x_2) = x_1 - x_2 = y_-$$

Partial derivatives:

$$\partial/\partial x_1 \ (x_1 - x_2) = 1$$
$$\partial/\partial x_2 \ (x_1 - x_2) = -1$$

- Derivatives are NOT symmetric
- Changing x_1 changes y_- identically, whereas changing x_2 changes y_- in the opposite direction (scales by −1)

Partial Derivatives of Multiplication

Function definition:

$$f_\times(x_1, x_2) = x_1 \times x_2 = y_\times$$

Partial derivatives:

$$\partial/\partial x_1 \ (x_1 \times x_2) = x_2$$
$$\partial/\partial x_2 \ (x_1 \times x_2) = x_1$$

- Scaling x_1 scales y by a factor of x_2, and vice versa
- The derivatives of $f_\times$ are themselves inputs to the function $f_\times$

Partial Derivatives of Division & Extending to Tensors

Partial Derivatives of Division

Function definition: $f\div(x_1,x_2) = x_1/x_2$

$$\partial/\partial x_1 \ (x_1/x_2) = 1/x_2$$
$$\partial/\partial x_2 \ (x_1/x_2) = -x_1/x_2^2$$

- Scaling x_1 changes y by a factor of $1/x_2$
- Changing x_2 scales y by a factor of $-x_1/x_2^2$
- Derivatives of f÷ are functions of the inputs to f÷

Extending Partial Derivatives to Tensors

- **Scaling Up:** Apply scalar derivative principles to full tensors
- **Order 1 Tensors:** Consider vectors u and v, each with k elements
- **Function Definition:** f(u, v) = w where w is the output tensor
- **Element-wise Derivatives:** Derive each triplet (u_i, v_i, w_i) at index i using scalar rules
- **Multi-valued Derivatives:** Rewrite derivatives to handle tensors, enabling complex operations on vector elements

Tensor Addition & Subtraction — Elementwise Derivatives

Tensor Addition and Derivatives

- **Vector Elementwise Addition Op:** `f(u,v) = u + v = w`
- **Derivative of f:**

```
∂w/∂u = ∂w/∂v = 1

  1 is a *vector* of same size as u and v
  containing all ones!
```

- **Interpretation:** Derivative is a tensor (vector) of ones, matching the size of u and v
- **Uniform Influence:** Each element in u and v contributes equally to w

Tensor Subtraction and Derivatives

- **Vector Elementwise Subtraction Op:** `f(u,v) = u − v = w`
- **Derivative w.r.t. u:**

```
∂w/∂u = 1     (vector of 1s)
```

- **Derivative w.r.t. v:**

```
∂w/∂v = −1    (vector of −1s)
```

- **Interpretation:** Each element of u and v contributes to w with equal magnitude but opposite signs

Tensor Multiplication & Division — Elementwise Derivatives

Tensor Multiplication and Derivatives

- **Vector Elementwise Mul Op:**

$$f_{\times}(u,v) = u \circ v = w \quad \text{(elementwise mul)}$$

- **Derivative w.r.t. u:**

$$\partial f / \partial u = v$$

- **Derivative w.r.t. v:**

$$\partial f / \partial v = u$$

- **Symmetry:** Derivatives are the other tensors themselves, similar to scalar multiplication

Tensor Division and Derivatives

- **Vector Elementwise Division Op:**

$$f_{\div}(u,v) = u / v = w \quad \text{(elementwise div)}$$

- **Derivative w.r.t. u:**

$$\partial w / \partial u = 1/v \quad \text{(element-wise reciprocal of v)}$$

- **Derivative w.r.t. v:**

$$\partial w / \partial v = -u/v^2$$

- These are elementwise ops, applying squaring, division, and negation to each element u_i and v_i from u and v, respectively.

FOUNDATION · VALUE CLASS

The Value Class — __slots__ & Memory Layout

Every node in the autograd graph is a Value instance. Two design decisions define its memory footprint: __slots__ (replaces Python's default __dict__ with fixed-offset fields) and four tightly-packed attributes.

__slots__ — skip the hash map

```
# Without __slots__ (Python default):
# Each object has __dict__ — a hash map:
#   PyObject header (16B)
#   + __dict__ ptr -> { 'data':float,
#                       'grad':float,
#                       '_children':tuple,
#                       '_local_grads':tuple }
#   Total: ~300 bytes per Value node
#
# WITH __slots__:
# Fixed-offset fields, no hash map:
#   PyObject header (16B)
#   + data | grad | _children | _local_grads
#   Total: ~80 bytes per Value node
#
# Savings: ~220 bytes/node
# 50,000 nodes/forward pass -> ~11 MB/step

__slots__ = ('data', 'grad',
             '_children', '_local_grads')
```

__init__ — four attributes annotated

```
def __init__(self, data,
             children=(),
             local_grads=()):

    self.data = data
    # The scalar value (float).
    # Computed during FORWARD pass.
    # e.g.  Value(3.0).data == 3.0

    self.grad = 0
    # dLoss / d(self).
    # Accumulated during BACKWARD pass.
    # Starts at 0; seed node gets 1.0.

    self._children = children
    # Tuple of Value nodes that PRODUCED
    # this node (inputs to this op).

    self._local_grads = local_grads
    # d(self) / d(child) for each child.
    # Paired 1-to-1 with _children.
```

Memory: ~80B/node with __slots__ vs ~300B without · 50 k nodes × 220B = ~11 MB saved per forward pass

Value Class — Python vs C++ Implementation

The Python Value class and the C++ struct serve the same role in the computation graph. Python uses __slots__ to approach C++ memory efficiency. C++ fields are always fixed-offset — no hash map to avoid.

```python
# Python — class Value
class Value:
    __slots__ = ('data', 'grad',
                 '_children',
                 '_local_grads')

    def __init__(self, data,
                 children=(),
                 local_grads=()):
        self.data         = data   # float
        self.grad         = 0      # float, init 0
        self._children    = children
        self._local_grads = local_grads

# Memory per node (with __slots__):
#   PyObject header:   16 B
#   data ptr:           8 B
#   grad ptr:           8 B
#   _children ptr:      8 B
#   _local_grads ptr:   8 B
#   Tuple objects:     ~32 B
#   ──────────────────────
#   Total:            ~80 B
```

```cpp
// C++ — struct Value
struct Value {
    float   data;          // 4 B
    float   grad = 0.0f;   // 4 B
    Value*  children[2];   // 16 B (2×ptr)
    float   local_grads[2];// 8 B
    uint8_t nchildren;     // 1 B
    // + 3 B padding (alignment)
    // Total:  ~36 B
};

// Key differences vs Python:
//
// 1. Fixed fields always — no __dict__.
//    Attribute access = fixed memory
//    offset from struct base ptr.
//
// 2. children[2] is INLINE array,
//    not a heap-allocated tuple.
//
// 3. local_grads[2] also inline.
//    Python: tuple objects on heap
//    (~32B each).
//
// 4. 36B vs ~80B — 2.2x more compact.
```

Attribute	Python Value	C++ struct Value	Notes
data	float	float	
grad	float (init 0)	float grad = 0.0f	
_children / children[2]	inputs	Value* children[2]	
_local_grads / local_grads[2]	d(self)/d(child)	float local_grads[2]	

Chain Rule — Manually Solved Worked Examples

The chain rule states: dL/dx = dL/dy × dy/dx. For multi-step computations, we multiply local derivatives back through each node in reverse order.

Example 1: f = (a + b) × c, a=2, b=3, c=4

```
Forward:
  s = a + b = 5
  f = s * c = 20

Local gradients:
  df/ds = c = 4
  df/dc = s = 5
  ds/da = 1
  ds/db = 1

Chain rule backward:
  dL/ds = dL/df * df/ds = 1 * 4 = 4
  dL/dc = dL/df * df/dc = 1 * 5 = 5
  dL/da = dL/ds * ds/da = 4 * 1 = 4
  dL/db = dL/ds * ds/db = 4 * 1 = 4

Result: da=4, db=4, dc=5   checkmark
```

Example 2: L = relu(a * b + c), a=2, b=3, c=1

```
Forward:
  p = a * b = 6
  s = p + c = 7
  L = relu(s) = 7   (s>0)

Local gradients:
  dL/ds = 1   (s>0 so relu gate=1)
  ds/dp = 1
  ds/dc = 1
  dp/da = b = 3
  dp/db = a = 2

Chain rule backward (L->s->p->a,b,c):
  dL/ds = 1 * 1 = 1
  dL/dp = dL/ds * 1 = 1
  dL/dc = dL/ds * 1 = 1
  dL/da = dL/dp * dp/da = 1 * 3 = 3
  dL/db = dL/dp * dp/db = 1 * 2 = 2

Result: da=3, db=2, dc=1   checkmark
```

Chain Rule — More Solved Examples (Sigmoid & Reuse)

Example 3: L = sigmoid(w*x + b), w=0.5, x=2, b=0.1

```
Forward:
  z = w * x = 1.0
  s = z + b = 1.1
  L = sigmoid(s) = 1/(1+e^-1.1)
    ~= 0.7503

Local gradients:
  dL/ds = sigmoid(s)*(1-sigmoid(s))
        = 0.7503*0.2497 ~= 0.1874
  ds/dz = 1,  ds/db = 1
  dz/dw = x = 2,  dz/dx = w = 0.5

Chain rule backward:
  dL/ds ~= 0.1874
  dL/dz = dL/ds * 1 = 0.1874
  dL/db = dL/ds * 1 = 0.1874
  dL/dw = dL/dz * dz/dw = 0.1874 * 2
        = 0.3747
  dL/dx = dL/dz * dz/dx = 0.1874 * 0.5
        = 0.0937

Result: dw=0.3747, dx=0.0937, db=0.1874
```

Example 4: Node Reuse — L = tanh(a) * a, a=1

```
Forward:
  t = tanh(a) ~= 0.7616
  L = t * a = 0.7616

  a feeds into L TWICE: once via t,
  once directly as the right operand.

Local gradients at L (multiply op):
  dL/dt = a = 1
  dL/da_direct = t ~= 0.7616

Local gradients at t (tanh op):
  dt/da = 1 - tanh(a)^2
        = 1 - 0.7616^2 ~= 0.4200

Chain rule — TWO PATHS to da:
  Path 1 (via t):
    dL/da_via_t = dL/dt * dt/da
                = 1 * 0.4200 = 0.4200
  Path 2 (direct):
    dL/da_direct = t ~= 0.7616

  da = 0.4200 + 0.7616 = 1.1816

Result: da ~= 1.1816   checkmark
```

Jacobian Matrices — From Scalar Derivatives to Tensor Derivatives

When inputs and outputs are vectors/tensors, the derivative becomes a matrix called the Jacobian. For an output of size m and input of size n, J is an m×n matrix where J[i][j] = ∂output_i / ∂input_j.

```
J = ∂f/∂x      where  J[i][j] = ∂f_i/∂x_j

For elementwise ops: J is DIAGONAL — only n non-zero entries out of n² total entries.
```

Addition f(A,B)=A+B

```
J(∂result/∂A) = Identity I

      A00 A01 A10 A11
R00:[ 1    0    0    0 ]
R01:[ 0    1    0    0 ]
R10:[ 0    0    1    0 ]
R11:[ 0    0    0    1 ]

J = I  (identity matrix)
```

Multiplication f(A,B)=A∘B

```
J(∂result/∂A) = diag(B_flat)

      A00 A01 A10 A11
R00:[ 2    0    0    0 ]
R01:[ 0    3    0    0 ]
R10:[ 0    0    4    0 ]
R11:[ 0    0    0    5 ]

J = diag([2,3,4,5]) = diag(B)
```

ReLU f(A)=max(0,A)

```
J(∂result/∂A) = diag(mask)

A = [[-1, 2],[ 3,-4]]
mask = (A>0) = [[0,1],[1,0]]

      A00 A01 A10 A11
R00:[ 0    0    0    0 ]
R01:[ 0    1    0    0 ]
R10:[ 0    0    1    0 ]
R11:[ 0    0    0    0 ]

J = diag(mask)
```

Key insight: Elementwise ops ALWAYS produce diagonal Jacobians. We never form the full matrix — we exploit this structure for O(n) backprop.

Full Jacobian: Elementwise Multiplication f(A,B) = A ∘ B

Using A=[[1,2],[3,4]] and B=[[2,3],[4,5]]. Each output element $R_{ir} = A_{ir} \times B_{ir}$ depends ONLY on A_{ir} — hence diagonal Jacobians.

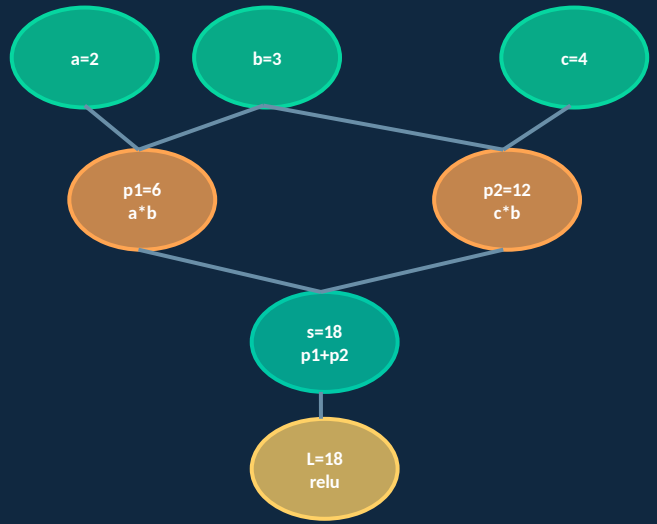
```
Full Jacobian ∂(result)/∂A  -- diagonal matrix with B values:

          A00  A01  A10  A11
  ∂R00: [  2    0    0    0  ]  # ∂(A00×B00)/∂A00 = B00 = 2
  ∂R01: [  0    3    0    0  ]  # ∂(A01×B01)/∂A01 = B01 = 3
  ∂R10: [  0    0    4    0  ]  # ∂(A10×B10)/∂A10 = B10 = 4
  ∂R11: [  0    0    0    5  ]  # ∂(A11×B11)/∂A11 = B11 = 5

  ∂(result)/∂A = diag(B_flat) = diag([2, 3, 4, 5])
```

```
Full Jacobian ∂(result)/∂B  -- diagonal matrix with A values:

          B00  B01  B10  B11
  ∂R00: [  1    0    0    0  ]  # ∂(A00×B00)/∂B00 = A00 = 1
  ∂R01: [  0    2    0    0  ]  # ∂(A01×B01)/∂B01 = A01 = 2
  ∂R10: [  0    0    3    0  ]  # ∂(A10×B10)/∂B10 = A10 = 3
  ∂R11: [  0    0    0    4  ]  # ∂(A11×B11)/∂B11 = A11 = 4

  ∂(result)/∂B = diag(A_flat) = diag([1, 2, 3, 4])
```

Computation Graph: DAG Structure & Why It Matters

A computation graph is a Directed Acyclic Graph (DAG). Nodes are values; edges encode dependencies. Backpropagation requires visiting nodes in reverse topological order — parents before children — to ensure all upstream gradients are accumulated before propagating.

DAG for: L = relu(a*b + c*b)




```
Topological sort (DFS post-order):
  Forward order:  [a, b, p1, c, p2, s, L]

Reversed (backprop order):
  [L, s, p2, c, p1, b, a]

Why reversed?
  When we process s, BOTH p1 and p2 must
  have already contributed their gradients.

  When we process b, BOTH p1 and p2 must
  have already run (b feeds into BOTH!).

Without topological order:
  We might process b BEFORE p1, and p2.
  Result: b.grad is incomplete -> WRONG.
```

DAG Rule: No cycles allowed (Acyclic). Each node can only be visited once. Reverse post-order guarantees all consumers run before producers.

BACKWARD IMPLEMENTATION

The backward() Function — Python DFS vs C++ Iterative

HOW IT WORKS

```
# Step 1: Build topological order via DFS
#
# Python: RECURSIVE DFS
#   Simple and readable.
#   Works for small graphs.
#   Risk: hits Python recursion limit
#   on very deep nets.
#
# C++ (autograd.hpp): ITERATIVE DFS
#   Uses explicit stack<Frame> to:
#   - Avoid recursion limit entirely
#   - Work without garbage collection
#   - Match systems-level constraints
#
# struct Frame { Value* v; int idx; };
# Both produce same post-order result:
#   leaves first, root last.
#
# Step 2: Seed gradient at loss node
#   self.grad = 1  (dL/dL = 1)
#
# Step 3: Reverse topological walk
#   For each node: accumulate grads
#   into children via chain rule:
#   child.grad += local_grad * v.grad
```

PYTHON IMPLEMENTATION

```
def backward(self):
    topo = []
    visited = set()

    def build_topo(v):
        if v not in visited:
            visited.add(v)
            for child in v._children:
                build_topo(child)
            topo.append(v)
            # append AFTER children
            # -> children come first

    build_topo(self)

    # Seed: dL/dL = 1
    self.grad = 1

    # Walk reversed topo order
    # (parents before children)
    for v in reversed(topo):
        for child, local_grad in zip(
            v._children,
            v._local_grads
        ):
            # Chain rule:
            # dL/dChild += dL/dV * dV/dChild
            child.grad += local_grad * v.grad
```

METHODOLOGY

The 6-Step Format Used for Every Operation

1 Forward Pass

Build the scalar graph. Record .data and local_grads for each node.

2 Chain Rule

Write the equation: $\partial L/\partial \text{input} = \partial L/\partial \text{output} \times \text{local\_grad}$.

3 Backward Pass

Topological sort → reverse → propagate seed gradient in reverse.

4 Jacobian

Full $\partial(\text{output})/\partial(\text{input})$ matrix for a concrete 2×2 tensor example.

5 Industry Implementation

The one-liner PyTorch/JAX/NumPy actually uses — no matrix formed.

6 Verify

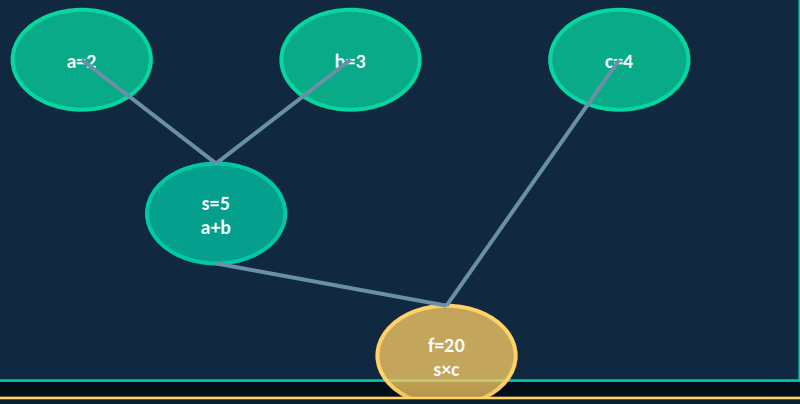
Confirm numerically against the analytic derivative.

FOUNDATION

Why Topological Sort? — Correct Gradient Accumulation

The chain rule requires that when we visit a node, ALL its consumers have already accumulated their gradient contributions. Topological sort guarantees this: every parent appears before its children, so reversing gives children-before-parents.

Computation Graph: (a + b) × c




```
Forward topo order (DFS post-order):
  [a, b, s, c, f]

Reversed (backprop order):
  [f, c, s, b, a]

s must be processed AFTER both f and t
have contributed to s.grad.

This is the correctness guarantee
of reverse-mode autodiff.
```

Rule:

Process a node only after ALL nodes that consume its output have already run their backward step. This ensures .grad is fully accumulated before being propagated further back.

ADD(A,B)

Addition — a + b

FORWARD PASS

```
a = 2,  b = 3
s = a + b
  s.data         = 5
  s.children     = [a, b]
  s.local_grads  = [1, 1]
    <- d/da(a+b) = 1,  d/db(a+b) = 1
```

CHAIN RULE

```
dL/da = dL/ds × ∂s/∂a = dL/ds × 1 = dL/ds
dL/db = dL/ds × ∂s/∂b = dL/ds × 1 = dL/ds
```

BACKWARD PASS

```
Topo reverse: [s, b, a]
1. s.grad = 1.0  <- seed
2. Process s:
   a.grad += 1 × 1.0 = 1.0
   b.grad += 1 × 1.0 = 1.0
3. Leaves: done
Result: a.grad = 1, b.grad = 1  ✔
Verify: ∂(a+b)/∂a = 1  ✔
```

JACOBIAN (2×2 TENSOR EXAMPLE)

```
A = [[1,2],[3,4]],  B = [[5,6],[7,8]]
result = A + B = [[6,8],[10,12]]

∂(result)/∂A: each Rᵢᵣ = Aᵢᵣ + Bᵢᵣ
  ∂Rᵢᵣ/∂Aᵢᵣ = 1,  ∂Rᵢᵣ/∂Aₖₗ = 0  (k,l != i,j)

      A00 A01 A10 A11
∂R00:[ 1   0   0   0 ]
∂R01:[ 0   1   0   0 ]
∂R10:[ 0   0   1   0 ]
∂R11:[ 0   0   0   1 ]

J = Identity I

grad_A = upstream * 1 = upstream
```

INDUSTRY IMPLEMENTATION

```
grad_A = upstream_grad
grad_B = upstream_grad

# Identity Jacobian = gradient passes through
# unchanged. No scaling, no transformation.
```

SUB(A,B)

Subtraction — a - b (= a + (-b) in code)

FORWARD PASS

```
a = 5,  b = 3
# CODE: __sub__ = self + (-other)
# Step 1: neg_b = b * -1  (MUL node)
#   neg_b.children=[b,-1], local_grads=[-1, b]
# Step 2: d = a + neg_b  (ADD node)
#   d.children=[a,neg_b], local_grads=[1,1]
# Net result: d.data = 5 + (-3) = 2
# Net: a.grad=+upstream, b.grad=-upstream
```

CHAIN RULE

```
dL/da = dL/dd × 1   = dL/dd
dL/db = dL/dd × (-1) = -dL/dd
# Via chain: upstream->add->neg->b
# b.grad += -1 * 1 * upstream = -upstream
```

BACKWARD PASS

```
Topo: [b, neg_b, a, d]  Rev: [d, a, neg_b, b]
1. d.grad = 1.0  <- seed
2. Process d (add):
   a.grad    += 1 × 1.0 = +1.0
   neg_b.grad += 1 × 1.0 = +1.0
3. Process a (leaf)
4. Process neg_b (mul by -1):
   b.grad += -1 × neg_b.grad = -1
Result: a.grad=+1, b.grad=-1  ✔
Same math as standalone sub, but 2 nodes.
```

JACOBIAN (2×2 TENSOR EXAMPLE)

```
A=[[5,6],[7,8]], B=[[1,2],[3,4]]
result = A + (-B) = A - B = [[4,4],[4,4]]

∂(result)/∂A:  J = Identity I (from add)
∂(result)/∂B:  J = -I (neg then add)
      B00 B01 B10 B11
∂R00:[-1    0    0    0 ]
∂R01:[ 0   -1    0    0 ]
∂R10:[ 0    0   -1    0 ]
∂R11:[ 0    0    0   -1 ]

grad_A=upstream; grad_B=-upstream
```

INDUSTRY IMPLEMENTATION

```
grad_A =  upstream_grad
grad_B = -upstream_grad

# NO standalone sub backward in code.
# Decomposes to: add(a, mul(b, -1))
# Autograd flows through both nodes.
```

Multiplication — a × b

FORWARD PASS

```
a = 3,  b = 4
p = a * b
  p.data        = 12
  p.children    = [a, b]
  p.local_grads = [b=4, a=3]
    <- d/da(a*b) = b,  d/db(a*b) = a
```

CHAIN RULE

```
dL/da = dL/dp × b
dL/db = dL/dp × a
```

BACKWARD PASS

```
Topo reverse: [p, b, a]
1. p.grad = 1.0  <- seed
2. Process p:
   a.grad += b × 1.0 = 4
   b.grad += a × 1.0 = 3
Result: a.grad = 4, b.grad = 3  ✔
Verify: ∂(a*b)/∂a = b = 4  ✔
```

JACOBIAN (2×2 TENSOR EXAMPLE)

```
A = [[1,2],[3,4]],  B = [[2,3],[4,5]]
result = A∘B = [[2,6],[12,20]]

∂(result)/∂A = diag(B_flat) = diag([2,3,4,5])
     A00 A01 A10 A11
∂R00:[ 2   0   0   0 ] <- B00=2
∂R01:[ 0   3   0   0 ] <- B01=3
∂R10:[ 0   0   4   0 ] <- B10=4
∂R11:[ 0   0   0   5 ] <- B11=5

∂(result)/∂B = diag(A_flat) = diag([1,2,3,4])
     B00 B01 B10 B11
∂R00:[ 1   0   0   0 ] <- A00=1
∂R11:[ 0   0   0   4 ] <- A11=4
```

INDUSTRY IMPLEMENTATION

```
grad_A = upstream_grad * B
grad_B = upstream_grad * A

# B values scale A's gradient.
# A values scale B's gradient.
# Diagonal Jacobian -> O(n) elementwise mul.
```

Division — a / b

FORWARD PASS

```
a = 6,  b = 3
q = a / b
  q.data        = 2
  q.children    = [a, b]
  q.local_grads = [1/b=0.333, -a/b²=-0.667]
    <- d/da(a/b) = 1/b
    <- d/db(a/b) = -a/b²
```

CHAIN RULE

```
dL/da = dL/dq × (1/b)
dL/db = dL/dq × (-a/b²)
```

BACKWARD PASS

```
Topo reverse: [q, b, a]
1. q.grad = 1.0  <- seed
2. Process q:
   a.grad += (1/3) × 1.0  =  0.333
   b.grad += (-6/9) × 1.0 = -0.667
Result: a.grad=0.333, b.grad=-0.667 ✔
Verify: ∂(a/b)/∂b = -a/b² = -0.667 ✔
```

JACOBIAN (2×2 TENSOR EXAMPLE)

```
A=[[2,4],[6,8]],  B=[[1,2],[3,4]]
result = A/B = [[2,2],[2,2]]

∂(result)/∂A = diag(1/B)
     A00 A01 A10 A11
∂R00:[ 1    0    0    0 ] <- 1/B00=1
∂R01:[ 0   .5    0    0 ] <- 1/B01=.5
∂R10:[ 0    0   .33   0 ] <- 1/B10=.33
∂R11:[ 0    0    0   .25] <- 1/B11=.25

∂(result)/∂B = diag(-A/B²)
∂R00:[-2    0    0    0 ] <- -A00/B00²
  J = diag(-A_flat/B_flat^2)
```

INDUSTRY IMPLEMENTATION

```
grad_A = upstream_grad / B
grad_B = -upstream_grad * A / (B*B)

# Never form the matrix.
# Both ops are pure elementwise.
```

Negation — neg(a) = -a

FORWARD PASS

```
a = 3
n = -a
  n.data         = -3
  n.children     = [a]
  n.local_grads  = [-1]
    <- d/da(-a)  = -1
```

CHAIN RULE

```
dL/da = dL/dn × (-1) = -dL/dn
```

BACKWARD PASS

```
Topo reverse: [n, a]
1. n.grad = 1.0  <- seed
2. Process n:
   a.grad += (-1) × 1.0 = -1
Result: a.grad = -1  ✔
Verify: d/da(-a) = -1  ✔

Unary op: only one child.
```

JACOBIAN (2×2 TENSOR EXAMPLE)

```
A = [[1,2],[3,4]]
result = -A = [[-1,-2],[-3,-4]]

J(∂result/∂A) = -I (negative identity)
      A00 A01 A10 A11
∂R00:[-1    0    0    0 ]
∂R01:[ 0   -1    0    0 ]
∂R10:[ 0    0   -1    0 ]
∂R11:[ 0    0    0   -1 ]

J = -I
grad_A = upstream * (-1) = -upstream
```

INDUSTRY IMPLEMENTATION

```
grad_A = -upstream_grad

# Negate the upstream gradient.
# Simplest possible backward pass.
```

ReLU — relu(a) = max(0, a)

FORWARD PASS

```
a = 2  (also check a=-1)
relu_pos = max(0, 2) = 2
  relu.data        = 2
  relu.children    = [a]
  relu.local_grads = [1]  <- a>0
For a=-1:
  relu_neg.data    = 0
  relu.local_grads = [0]  <- a<=0
```

CHAIN RULE

```
dL/da = dL/drelu × (1 if a>0 else 0)
      = dL/drelu × (a > 0)
```

BACKWARD PASS

```
Topo reverse: [relu, a]
1. relu.grad = 1.0  <- seed
2. a>0: a.grad += 1 × 1.0 = 1.0
   a<0: a.grad += 0 × 1.0 = 0.0
Result: gate passes or blocks gradient

ReLU = a gradient GATE:
  Open (a>0):  gradient flows through
  Closed (a≤0): gradient is zeroed
```

JACOBIAN (2×2 TENSOR EXAMPLE)

```
A = [[-1,2],[3,-2]]
result = relu(A) = [[0,2],[3,0]]
mask = (A>0) = [[0,1],[1,0]]

J(∂result/∂A) = diag(mask)
      A00 A01 A10 A11
∂R00:[ 0    0    0    0 ] <- A00<0
∂R01:[ 0    1    0    0 ] <- A01>0
∂R10:[ 0    0    1    0 ] <- A10>0
∂R11:[ 0    0    0    0 ] <- A11<0

J = diag(mask)
```

INDUSTRY IMPLEMENTATION

```
grad_A = upstream_grad * (A > 0)

# Boolean mask * upstream.
# Dead neurons: A<=0 gives zero grad.
```

Logarithm — log(a) (natural log)

FORWARD PASS

```
a = 2
l = log(a) ~= 0.693
  l.data        ~= 0.693
  l.children    = [a]
  l.local_grads = [1/a = 0.5]
    <- d/da(log a) = 1/a
```

CHAIN RULE

```
dL/da = dL/dl × (1/a)
```

BACKWARD PASS

```
Topo reverse: [l, a]
1. l.grad = 1.0  <- seed
2. Process l:
   a.grad += (1/2) × 1.0 = 0.5
Result: a.grad = 0.5  ✔
Verify: d/da(log a)|a=2 = 1/2 = 0.5 ✔

Note: requires a > 0.
```

JACOBIAN (2×2 TENSOR EXAMPLE)

```
A = [[1,2],[3,4]]
result = log(A)
~= [[0,0.693],[1.099,1.386]]

∂(result)/∂A = diag(1/A_flat)
     A00 A01 A10 A11
∂R00:[ 1    0    0    0 ] <- 1/A00=1
∂R01:[ 0   .5    0    0 ] <- 1/A01=.5
∂R10:[ 0    0   .33   0 ] <- 1/A10=.33
∂R11:[ 0    0    0   .25] <- 1/A11=.25

J = diag(1/A_flat)
```

INDUSTRY IMPLEMENTATION

```
grad_A = upstream_grad / A

# Divide upstream by A element-wise.
# Gradient grows as A -> 0 (unstable).
```

Exponential — exp(a)

FORWARD PASS

```
a = 1
e = exp(1)  ~= 2.718
  e.data         ~= 2.718
  e.children     = [a]
  e.local_grads = [exp(a)] = [2.718]
    <- d/da(exp a) = exp(a) = result!
```

CHAIN RULE

```
dL/da = dL/de × exp(a) = dL/de × result
```

BACKWARD PASS

```
Topo reverse: [e, a]
1. e.grad = 1.0   <- seed
2. Process e:
   a.grad += 2.718 × 1.0 = 2.718
Result: a.grad ~= 2.718  ✔

Key: result = exp(a) is kept from forward.
No recomputation needed.
```

JACOBIAN (2×2 TENSOR EXAMPLE)

```
A = [[0,1],[2,3]]
result = exp(A)
~= [[1,2.718],[7.389,20.086]]

∂(result)/∂A = diag(result_flat)
      A00  A01  A10  A11
∂R00:[ 1    0    0    0 ] <- e^0=1
∂R01:[ 0  2.72   0    0 ] <- e^1
∂R10:[ 0    0  7.39  0 ] <- e^2
∂R11:[ 0    0    0 20.09] <- e^3

J = diag(exp(A_flat)) = diag(result)
```

INDUSTRY IMPLEMENTATION

```
grad_A = upstream_grad * result

# result = exp(a) from forward pass.
# Its own derivative is itself!
```

Power — a^e

FORWARD PASS

```
a = 3,  e = 2
p = a^e = 9
  p.data        = 9
  p.children    = [a]
  p.local_grads = [e * a^(e-1)]
                = [2 * 3^1] = [6]
    <- d/da(a^e) = e * a^(e-1)
```

CHAIN RULE

```
dL/da = dL/dp × e × a^(e-1)
```

BACKWARD PASS

```
Topo reverse: [p, a]
1. p.grad = 1.0  <- seed
2. Process p:
   a.grad += 6 × 1.0 = 6
Result: a.grad = 6  ✔
Verify: d/da(a^2)|a=3 = 2*3 = 6  ✔
```

JACOBIAN (2×2 TENSOR EXAMPLE)

```
A=[[1,2],[3,4]],  e=2
result = A^2 = [[1,4],[9,16]]

∂(result)/∂A = diag(e * A^(e-1))
     A00 A01 A10 A11
∂R00:[ 2    0    0    0 ] <- 2*A00=2*1
∂R01:[ 0    4    0    0 ] <- 2*A01=2*2
∂R10:[ 0    0    6    0 ] <- 2*A10=2*3
∂R11:[ 0    0    0    8 ] <- 2*A11=2*4

J = diag(2*A_flat) = diag([2,4,6,8])
```

INDUSTRY IMPLEMENTATION

```
grad_A = upstream_grad * e * A**(e-1)

# Standard power rule.
# e is a constant (not a tensor).
```

Tanh — tanh(a) = (e^2a - 1)/(e^2a + 1)

FORWARD PASS

```
a = 1
t = tanh(1)  ~= 0.762
  t.data         ~= 0.762
  t.children     = [a]
  t.local_grads  = [1 - tanh(a)^2]
                 = [1 - 0.580] = [0.420]
    <- d/da(tanh) = 1 - tanh²
```

CHAIN RULE

```
dL/da = dL/dt × (1 - tanh(a)²)
      = dL/dt × (1 - result²)
```

BACKWARD PASS

```
Topo reverse: [t, a]
1. t.grad = 1.0  <- seed
2. Process t:
   a.grad += 0.420 × 1.0 = 0.420
Result: a.grad ~= 0.420  ✔

Max gradient = 1 at a=0.
Saturates to 0 as |a| -> inf.
```

JACOBIAN (2×2 TENSOR EXAMPLE)

```
A = [[0,1],[-1,2]]
result = tanh(A)
~= [[0,0.762],[-0.762,0.964]]
1-result^2
~= [[1,0.420],[0.420,0.071]]

J(∂result/∂A) = diag(1-result^2)
     A00 A01 A10 A11
∂R00:[ 1    0    0    0 ] <- a=0
∂R01:[ 0   .42   0    0 ]
∂R10:[ 0    0   .42   0 ]
∂R11:[ 0    0    0  .07]
```

INDUSTRY IMPLEMENTATION

```
grad_A = upstream_grad * (1 - result**2)

# result = tanh(a) from forward.
# Peak gradient=1 at a=0; saturates.
```

Sigmoid — σ(a) = 1/(1+exp(-a))

FORWARD PASS

```
a = 1
sg = 1/(1+exp(-1)) ~= 0.731
  sg.data        ~= 0.731
  sg.children    = [a]
  sg.local_grads = [sg × (1 - sg)]
                 ~= [0.731 × 0.269] = [0.197]
    <- d/da sigma = sigma × (1-sigma)
```

CHAIN RULE

```
dL/da = dL/dsg × sigma(a)(1 - sigma(a))
      = dL/dsg × result × (1 - result)
```

BACKWARD PASS

```
Topo reverse: [sg, a]
1. sg.grad = 1.0  <- seed
2. Process sg:
   a.grad += 0.197 × 1.0 = 0.197
Result: a.grad ~= 0.197  ✔

Max gradient = 0.25 at a=0.
Same saturation problem as tanh.
```

JACOBIAN (2×2 TENSOR EXAMPLE)

```
A = [[0,1],[-1,2]]
result = sg(A) ~= [[0.5,0.731],[0.269,0.880]]
result*(1-result) ~=
  [[0.25,0.197],[0.197,0.105]]

J(∂result/∂A) = diag(result*(1-result))
     A00  A01  A10  A11
∂R00:[.25   0    0    0 ]
∂R01:[ 0  .197   0    0 ]
∂R10:[ 0    0  .197   0 ]
∂R11:[ 0    0    0  .105]
```

INDUSTRY IMPLEMENTATION

```
grad_A = upstream_grad * result * (1 - result)

# result already in memory from forward.
# Peak gradient 0.25 at a=0.
```

Node Reuse — a * a (same node as both inputs)

When a single node feeds into an operation as BOTH inputs, the backward pass must accumulate gradient contributions from BOTH paths. The += operator ensures both contributions land on the same .grad before propagation.

```
a = 3
p = a * a   <- a used TWICE
  p.data        = 9
  p.children    = [a, a]  <- same object!
  p.local_grads = [a, a] = [3, 3]
    <- d(a*a)/da_left=a,  d(a*a)/da_right=a
```

```
dL/da = dL/dp × dp/da_left  +  dL/dp × dp/da_right
      = dL/dp × a  +  dL/dp × a
      = 2a × dL/dp
```

```
Topo reverse: [p, a]
1. p.grad = 1.0  <- seed
2. Process p:
   a.grad += local_grads[0] × p.grad = 3
   a.grad += local_grads[1] × p.grad = 3
   (+=  accumulates, never overwrites)
3. a.grad = 6  checkmark
Verify: d/da(a^2) = 2a = 6  checkmark
```

```
A = [[1,2],[3,4]]
p = A * A = A^2 = [[1,4],[9,16]]

J(dp/dA):
  Each p_i = a_i^2,  depends only on a_i.
  dp_i/da_j = 0    for i != j
  dp_i/da_i = 2*a_i

J = diag(2A) = diag([2,4,6,8])

upstream = [[2,1],[3,2]]
grad_A = upstream * 2A = [[4,4],[18,16]]
```

```
# Implementation with += (critical!)
a.grad = 0
a.grad += B * p.grad   # contribution 1
a.grad += A * p.grad   # contribution 2 (A==B)

# Simplified:
grad_A = upstream_grad * 2 * A
# [[4,4],[18,16]]
```

Composed Example — (a + b) * c

Two operations chained: the addition result feeds into multiplication. Gradient flows backward through both operations via repeated chain rule application.

```
a=2,  b=3,  c=4

Step 1:  s = a + b
  s.data = 5,  local_grads=[1, 1]

Step 2:  f = s * c
  f.data = 20,  local_grads=[c=4, s=5]

Graph: [a=2] [b=3]  [c=4]
           \  /    /
           [s=5]  /
             \   /
             [f=20]
```

```
dL/ds = dL/df × df/ds = dL/df × c = 4
dL/dc = dL/df × df/dc = dL/df × s = 5
dL/da = dL/ds × ds/da = 4 × 1 = 4
dL/db = dL/ds × ds/db = 4 × 1 = 4
```

```
Topo: [a,b,s,c,f]  Reverse: [f,c,s,b,a]

1. f.grad = 1.0  <- seed
2. Process f (mul):
   s.grad += 4 × 1.0 = 4
   c.grad += 5 × 1.0 = 5
3. Process c (leaf): done
4. Process s (add):
   a.grad += 1 × 4 = 4
   b.grad += 1 × 4 = 4
5-6. Leaves: done

Result: a.grad=4, b.grad=4, c.grad=5  ✔
```

Node Reuse in Larger Graph — f = (s + c) * s

s = a + b feeds into f TWICE — once via t = (s + c) and once directly. Gradient accumulates from BOTH paths before s can backpropagate further. This is why topological sort is essential.

```
a=4,  b=6,  c=1
s = a + b = 10
t = s + c = 11   <- s appears here
f = t * s = 110  <- s appears here too

Graph:
  [a=4] [b=6]    [c=1]
     \  /    \  /
     [s=10]  [t=11]
        \    /
         [f=110]
  s feeds into BOTH t and f
```

```
s receives gradient from two paths:

dL/ds =  dL/df × df/dt × dt/ds  (via t)
       + dL/df × df/ds           (direct)
= 1 × t × 1  +  1 × s
= 11         +  10  = 21
```

```
Topo: [a,b,s,c,t,f]  Reverse: [f,t,c,s,b,a]

1. f.grad = 1.0
2. Process f (mul, children=[t,s]):
   t.grad += s × f.grad = 10 × 1 = 10
   s.grad += t × f.grad = 11 × 1 = 11  1st
3. Process t (add, children=[s,c]):
   s.grad += 1 × t.grad = 10         2nd
   c.grad += 1 × t.grad = 10
4. Process c (leaf)
5. Process s  <- s.grad = 11+10 = 21  FULL
   a.grad += 1 × 21 = 21
   b.grad += 1 × 21 = 21

Result: a=21, b=21, c=10  ✔
```

Cross-Entropy Loss — -log(softmax(z)[target])

```
z = [1, 2, 3],  target = 2,  max_z = 3

Step 1: shifted = z - max_z = [-2,-1,0]
Step 2: exps[i] = exp(shifted[i])
        = [0.135, 0.368, 1.000]
Step 3: total   = sum(exps) ~= 1.503
Step 4: prob[i] = exps[i]/total
        = [0.090, 0.245, 0.665]
Step 5: loss = -log(prob[2]) ~= 0.408
```

```
Unified gradient shortcut:
  dL/dz[i] = prob[i] - 1[i == target]

z[0].grad = 0.090 - 0 = +0.090
z[1].grad = 0.245 - 0 = +0.245
z[2].grad = 0.665 - 1 = -0.335  (target)

  Target logit grad < 0 (push UP)
  Other logits grad > 0 (push DOWN)
```

```
Softmax Jacobian — DENSE (not diagonal):
  d(softmax_i)/dz_j = p_i(delta_ij - p_j)

3x3 Jacobian with p=[0.090,0.245,0.665]:
        z0       z1       z2
p0: [ 0.082   -0.022   -0.060 ]
p1: [-0.022    0.185   -0.163 ]
p2: [-0.060   -0.163    0.223 ]

NEVER computed in practice.
O(vocab^2) memory is prohibitive.
```

```
def cross_entropy_backward(probs, target):
    grad = probs.copy()
    grad[target] -= 1   # subtract 1
    return grad / batch_size

# Result: [0.090, 0.245, -0.335]
# Dense Jacobian never formed.
# O(vocab) not O(vocab^2).
```

SHAPE-TRANSFORMING OPS

Additional Operations — Reshape, Broadcast, Concat, Transpose

These operations change shape without changing values. Their gradients require reshaping or accumulating the upstream gradient to match input shapes.

Reshape / View

```
Forward:  output = A.reshape(new_shape)
Backward: grad_A = upstream.reshape(A.shape)

Ex: A = [[1,2],[3,4]]  shape=(2,2)
    view: [1,2,3,4]  shape=(4,)
    upstream = [g0,g1,g2,g3]
    grad_A = [[g0,g1],[g2,g3]]

Jacobian: identity permutation matrix.
```

Broadcast

```
Forward:  output = A.broadcast_to(shape)
Backward: grad_A = upstream.sum(along broadcast dims)

Ex: A = [1,2]  broadcast to [[1,2],[1,2],[1,2]]
    upstream = [[g0,g1],[g2,g3],[g4,g5]]
    grad_A = [g0+g2+g4, g1+g3+g5] = [9,6]

Must SUM gradients back to original size.
```

Concatenate (cat)

```
Forward:  output = cat([A,B], dim=0)
Backward: split upstream back to each input

Ex: A=[1,2] B=[3,4,5]  cat=[1,2,3,4,5]
    upstream = [g0,g1,g2,g3,g4]
    grad_A = upstream[:2] = [g0,g1]
    grad_B = upstream[2:] = [g2,g3,g4]

Split gradient by original sizes.
```

Transpose (T)

```
Forward:  output = A.T  (or permute dims)
Backward: grad_A = upstream.T

Ex: A = [[1,2],[3,4]]  shape=(2,2)
    A.T = [[1,3],[2,4]]
    upstream = [[g0,g1],[g2,g3]]
    grad_A = upstream.T = [[g0,g2],[g1,g3]]

Reverse the permutation.
```

KEY INSIGHT

Why All Elementwise Jacobians Are Diagonal

For any elementwise op z_i = f(u_i):

```
dz_i/du_j = 0      whenever i != j   (output i does NOT depend on input j)
dz_i/du_i = f'(u_i)                  (only the diagonal entry is non-zero)
```

Diagonal Structure

```
J = diag([f'(u0), f'(u1), ...,
f'(un)])
```

Only n non-zero entries out of n^2.
Off-diagonal zeros are never stored.

O(n) Jacobian-Vector Product

```
J^T x g = diag(f'(u)) x g
        = f'(u) * g
```

One elementwise multiply replaces an
n^2 matrix multiply completely.

Universal Industry Pattern

```
grad_input = upstream *
local_deriv
```

Every library follows this. No loop,
no matrix, no alloc — O(n) time.

Exception: Softmax has a DENSE Jacobian (normalization couples every output to every input). It is never formed explicitly — backward uses the O(n) shortcut: grad = probs then grad[target] -= 1.

REFERENCE

Quick Reference — All Operations at a Glance

Op	Industry Shortcut	Jacobian Structure	local_grad
a+b	grad_A=upstream; grad_B=upstream	Identity I	[1, 1]
a-b	grad_A=upstream; grad_B=-upstream	I and -I	[1, -1]
a*b	grad_A=upstream*B; grad_B=upstream*A	diag(B)/diag(A)	[b, a]
a/b	grad_A=upstream/B; grad_B=-up*A/B^2	diag(1/B)...	[1/b,-a/b^2]
neg(a)	grad_A=-upstream	-I	[-1]
relu(a)	grad_A=upstream*(A>0)	diag(mask)	[1(a>0)]
log(a)	grad_A=upstream/A	diag(1/A)	[1/a]
exp(a)	grad_A=upstream*result	diag(result)	[exp(a)]
a^e	grad_A=upstream*e*A^(e-1)	diag(e*A^(e-1))	[e*a^(e-1)]
tanh(a)	grad_A=upstream*(1-result^2)	diag(1-res^2)	[1-tanh^2(a)]
sigmoid	grad_A=upstream*result*(1-result)	diag(sg*(1-sg))	[sg*(1-sg)]
a*a	grad_A=upstream*2*A	diag(2A)	[a,a]->2a
softmax +CE	grad=probs; grad[t]-=1	Dense (n^2)	never formed

Extended Operations Reference Table

Op	In A	In B	Output	up_grad	grad_A	grad_B	Notes
broadcast	[[1,2]]	N/A	[[1,2]]*3	[[2,1],[3,2],[4,3]]	[[3],[6]]	N/A	Sum along broadcasted dimensions
view	[[1,2],[3,4]]	N/A	[1,2,3,4]	[2,1,3,2]	[[2,1],[3,2]]	N/A	Reshape gradient to match original shape
triu	[[1,2],[3,4]]	N/A	[[1,2],[0,4]]	[[2,1],[3,2]]	[[2,1],[0,2]]	N/A	Upper triangular keeps grad, lower is zero
tril	[[1,2],[3,4]]	N/A	[[1,0],[3,4]]	[[2,1],[3,2]]	[[2,0],[3,2]]	N/A	Lower triangular keeps grad, upper is zero
index	[1,2,3,4,5]	[0,2,4]	[1,3,5]	[10,20,30]	[10,0,20,0,30]	N/A	Gradient only flows to selected indices
squeeze	[[[1,2]]]	N/A	[1,2]	[2,1]	[[[2,1]]]	N/A	Add back removed dimensions
unsqueeze	[1,2]	N/A	[[1,2]]	[[2,1]]	[2,1]	N/A	Remove added dimensions
permute	[[1,2],[3,4]]	N/A	[[1,3],[2,4]]	[[2,1],[3,2]]	[[2,3],[1,2]]	N/A	Reverse the permutation
flatten	[[1,2],[3,4]]	N/A	[1,2,3,4]	[2,1,3,2]	[[2,1],[3,2]]	N/A	Reshape back to original shape
min	[3,1,4,1,5]	N/A	1	1	[0,1,0,0,0]	N/A	Grad flows only to minimum index
max	[3,1,4,1,5]	N/A	5	1	[0,0,0,0,1]	N/A	Grad flows only to maximum index
mean	[1,2,3,4]	N/A	2.5	1	[.25,.25,.25,.25]	N/A	Grad distributed equally: 1/n per element
mask	[1,2,3,4]	[T,F,T,F]	[1,0,3,0]	[2,1,3,2]	[2,0,3,0]	[0,0,0,0]	Gradient flows only through mask=True

Optional Operations — Stack, Chunk, Fill, masked_fill

Op	In A	In B	Output	up_grad	grad_A	grad_B	Notes
stack	[1,2] & [3,4]	N/A	[[1,2],[3,4]]	[[2,1],[3,2]]	[2,1] & [3,2]	N/A	Split gradient back to each input
chunk	[1,2,3,4]	N/A	[1,2] & [3,4]	[2,1] & [3,2]	[2,1,3,2]	N/A	Concatenate gradients along split dim
fill	[1,2,3]	5 (fill val)	[5,5,5]	[2,1,3]	[0,0,0]	6	Input gets zero, fill value gets sum
mask	[1,2,3,4]	[T,F,T,F]	[1,0,3,0]	[2,1,3,2]	[2,0,3,0]	[0,0,0,0]	Gradient flows only through mask=True
masked_fill	[1,2,3,4]	[T,F,T,F],9	[9,2,9,4]	[2,1,3,2]	[0,1,0,2]	5	A gets grad where mask=False, fill gets sum
clamp	[-2,1,3,5]	min=0,max=4	[0,1,3,4]	[2,1,3,2]	[0,1,3,0]	N/A	Grad blocked at clamped values (0 there)
abs	[-3,2,-1]	N/A	[3,2,1]	[1,2,1]	[-1,2,-1]	N/A	sign(A) * upstream; zero at A=0 (subgrad)
softmax	[1,2,3]	N/A	[.09,.24,.67]	[g0,g1,g2]	p*(g-p·g)	N/A	Dense Jacobian; in practice: use CE shortcut

Jacobian Structures Across All Operations

Every operation falls into one of three Jacobian classes. This classification determines the O(n) backward shortcut.

Class 1: Identity / Scaled Identity

Ops: add, subtract, neg, reshape, view, flatten

```
J = I  or  J = c*I

Off-diagonal: all zero
Diagonal:     all 1 (or all c)

Backward: grad = upstream (or
c*upstream)
```

Class 2: Diagonal (f'(a) varies per element)

Ops: mul, div, relu, log, exp, pow, tanh, sigmoid, abs, clamp

```
J = diag([f'(a0), f'(a1), ...,
f'(an)])

Off-diagonal: all zero
Diagonal:     f'(a_i) for each i

Backward: grad = upstream * f'(A)
```

Class 3: Dense Jacobian (all entries non-zero)

Ops: softmax, matmul (off-diagonal couplings)

```
J = full n×n matrix (or n×m)

All entries potentially non-zero.
O(n^2) to form. Never formed in
practice.

Backward: special O(n) shortcut
used
```

Broadcast, index, transpose, concat: shape-manipulation ops. Backward = reshape/sum/split — never form Jacobian.

Python Dunder Ops — How They Compose Primitives

MicroGPT defines only a few primitive ops with their own backward(). All other arithmetic operators are Python dunder methods that COMPOSE those primitives — so autograd flows automatically through the composed graph.

Method	Code (MicroGPT)	Graph Decomposition	Primitives Used
`__neg__(self)`	`return self * -1`	`MUL(self, -1)`	`mul`
`__radd__(self, other)`	`return self + other`	`ADD(self, other)`	`add`
`__sub__(self, other)`	`return self + (-other)`	`ADD(self, MUL(other, -1))`	`add + mul`
`__rsub__(self, other)`	`return other + (-self)`	`ADD(other, MUL(self, -1))`	`add + mul`
`__rmul__(self, other)`	`return self * other`	`MUL(self, other)`	`mul`
`__truediv__(self, other)`	`return self * other**-1`	`MUL(self, POW(other, -1))`	`mul + pow`
`__rtruediv__(self, other)`	`return other * self**-1`	`MUL(other, POW(self, -1))`	`mul + pow`

Key insight: No new backward() needed. Composing primitives means autograd handles everything automatically — this is the elegance of the design.

__NEG__

__neg__ — -self = self * -1

*Negation has NO dedicated backward(). Python translates -a into a.__neg__() which returns self * -1, creating a MUL node in the graph. Autograd through the MUL node produces the correct gradient automatically.*

```
# MicroGPT source:
def __neg__(self):
    return self * -1

# What this creates in the graph:
#
#   neg_a = MUL(self=a, other=Value(-1))
#
#   neg_a.data        = a.data * -1
#   neg_a.children    = [a, Value(-1)]
#   neg_a.local_grads = [other=-1, self=a.data]
#
# Backward through MUL:
#   a.grad        += -1      * neg_a.grad  = -upstream
#   Value(-1).grad += a.data * neg_a.grad  (leaf, ignored)
#
# Net result: a.grad = -upstream  (correct!)
```

```
# Worked example: a = 3
a = Value(3)
neg_a = -a          # calls __neg__ -> a * -1

# Graph created:
#   [a=3] ---> [MUL=neg_a, data=-3]
#   [Value(-1)] -/
#
# Forward:  neg_a.data = 3 * -1 = -3
#
# Backward (seed=1 at neg_a):
#   local_grad_a = other = -1
#   a.grad += -1 * 1.0 = -1  checkmark
#
# Verify: d/da(-a) = -1  checkmark
```

```
# Jacobian perspective (A = [[1,2],[3,4]]):
# result = A * -1 = [[-1,-2],[-3,-4]]
#
# J(dresult/dA) = diag(-1, -1, -1, -1) = -I
#
```

__RADD__

__radd__ — other + self (reflected add)

Python calls __radd__ when the LEFT operand doesn't know how to add (e.g. it's a plain int or float). The result is identical to __add__ — same ADD node, same backward. No new primitive needed.

```
# MicroGPT source:
def __radd__(self, other):
    return self + other     # delegates to __add__

# When does Python call __radd__?
#   3 + tensor   --> int(3).__add__(tensor) fails
#                --> tensor.__radd__(3) called
#
# What __radd__ does:
#   return self + other
#   = self.__add__(other)
#   other is wrapped: Value(3) if it's a scalar
#
# Graph created: exactly the same as ADD
#   result.children    = [self, Value(other)]
#   result.local_grads = [1, 1]
#
# Backward:
#   self.grad        += 1 * upstream
#   Value(other).grad += 1 * upstream  (leaf, ignored)
```

```
# Worked example: 5 + a  (int + Value)
a = Value(3)
result = 5 + a      # calls a.__radd__(5)

# Equivalent to:
result = a + 5      # same graph!

# Graph:
#   [a=3] -----> [ADD=result, data=8]
#   [Value(5)] -/
#
# Forward:  result.data = 3 + 5 = 8
#
# Backward (seed=1 at result):
#   a.grad        += 1 * 1.0 = +1  checkmark
#   Value(5).grad += 1 * 1.0 = +1  (leaf, ignored)
#
# Verify: d/da(5+a) = 1  checkmark
# Verify: d/da(a+5) = 1  checkmark (same!)
```

Commutativity: addition is commutative so __radd__ = __add__ exactly. The Python dispatch mechanism (r-methods) is purely about which object's method gets called — not about math.

__SUB__

__sub__ — self - other = self + (-other)

Subtraction decomposes into ADD + NEG (which is itself MUL by -1). Two nodes are created. The chain rule flows through both — no dedicated subtraction backward() needed.

```
# MicroGPT source:
def __sub__(self, other):
    return self + (-other)    # NEG then ADD

# Step 1: neg_other = -other  -> other.__neg__()
#   = other * -1   (MUL node)
#   neg_other.children    = [other, Value(-1)]
#   neg_other.local_grads = [-1, other.data]
#
# Step 2: result = self + neg_other   (ADD node)
#   result.children    = [self, neg_other]
#   result.local_grads = [1, 1]
#
# Full graph:
#   [self] ----------> [ADD = result]
#   [other] -> [MUL=neg_other] -/
```

```
# Worked example: a=5, b=3  ->  a - b
a, b = Value(5), Value(3)
result = a - b

# Forward:
#   neg_b.data = 3 * -1 = -3
#   result.data = 5 + (-3) = 2
#
# Backward (seed=1 at result):
#   1. result (ADD):  a.grad      += 1 * 1 = +1
#                     neg_b.grad  += 1 * 1 = +1
#   2. neg_b (MUL):  b.grad  += -1 * neg_b.grad
#                                = -1 * 1 = -1
#
# Result: a.grad=+1, b.grad=-1  checkmark
# Verify: d/da(a-b)=+1, d/db(a-b)=-1  checkmark
```

```
# Why 2 nodes instead of 1?
#   Graph: self ---> ADD ---> result
#          other -> MUL(x-1) -/
#
# Jacobians through the graph:
#   ADD node: dADD/d(self) = I,  dADD/d(neg_other) = I
#   MUL node: dMUL/d(other) = diag(-1) = -I
```

__RSUB__

__rsub__ — other - self = other + (-self)

Called when the left operand (a plain int/float) fails its __sub__. Here 'self' is the VALUE tensor and 'other' is the scalar. The sign of self's gradient is NEGATIVE because self is being negated.

```
# MicroGPT source:
def __rsub__(self, other):
    return other + (-self)    # NEG self, then ADD

# When called:  5 - tensor  -> tensor.__rsub__(5)
#
# Step 1: neg_self = -self  -> self.__neg__()
#   = self * -1   (MUL node)
#   neg_self.children    = [self, Value(-1)]
#   neg_self.local_grads = [-1, self.data]
#
# Step 2: result = other + neg_self  (ADD node)
#   other wrapped as Value(other) if scalar
#   result.children    = [Value(other), neg_self]
#   result.local_grads = [1, 1]
#
# Full graph:
#   [Value(other)] ---------> [ADD = result]
#   [self] -> [MUL=neg_self] -/
```

```
# Worked example: 5 - a  (int - Value)
a = Value(3)
result = 5 - a      # calls a.__rsub__(5)

# Forward:
#   neg_a.data  = 3 * -1 = -3
#   result.data = 5 + (-3) = 2
#
# Backward (seed=1 at result):
#   1. result (ADD):
#      Value(5).grad += 1 * 1 = +1  (ignored)
#      neg_a.grad    += 1 * 1 = +1
#   2. neg_a (MUL by -1):
#      a.grad += -1 * neg_a.grad = -1 * 1 = -1
#
# a.grad = -1
# Verify: d/da(5-a) = -1  checkmark
#
# Compare __sub__:  (a-5) -> a.grad = +1
# Compare __rsub__: (5-a) -> a.grad = -1  (flipped!)
```

Key distinction __sub__ vs __rsub__:

```
__sub__(a, b)  = a - b  ->  a.grad = +upstream,  b.grad = -upstream
__rsub__(a, b) = b - a  ->  a.grad = -upstream,  b.grad = +upstream  (a and b roles SWAP)
```

__RMUL__

__rmul__ — other * self (reflected multiply)

*Called when a scalar is on the LEFT side of multiplication (e.g. 3 * tensor). Delegates directly to __mul__ — multiplication is commutative so the graph and gradients are identical.*

```
# MicroGPT source:
def __rmul__(self, other):
    return self * other    # delegates to __mul__

# When called:  3 * tensor  -> tensor.__rmul__(3)
#
# Result: MUL(self, Value(other))
#   result.data        = self.data * other
#   result.children    = [self, Value(other)]
#   result.local_grads = [other, self.data]
#
# Backward:
#   self.grad         += other     * upstream
#   Value(other).grad += self.data * upstream   (leaf)
#
# This is IDENTICAL to self * other (__mul__)
# Multiplication is commutative: a*b = b*a
```

```
# Worked example: 3 * a  (scalar * Value)
a = Value(4)
result = 3 * a      # calls a.__rmul__(3)

# Equivalent to:
result2 = a * 3     # same graph!

# Graph:
#   [a=4] --------> [MUL=result, data=12]
#   [Value(3)] --/
#
# Forward: result.data = 4 * 3 = 12
#
# Backward (seed=1):
#   local_grad_a = other = 3
#   a.grad += 3 * 1.0 = 3  checkmark
#
# Verify: d/da(3*a) = 3  checkmark
# Verify: d/da(a*3) = 3  checkmark (same!)
```

__radd__ vs __rmul__ — the symmetric pair:

Both just delegate to their forward counterpart. Both work because add and mul are commutative.
__rsub__ and __rtruediv__ CANNOT do this — subtraction and division are NOT commutative (a-b ≠ b-a, a/b ≠ b/a).

__TRUEDIV__

__truediv__ — self / other = self * other**-1

Division decomposes into POW(other, -1) then MUL. Two nodes in the graph. The chain rule threads through both automatically — no dedicated division backward() needed.

```
# MicroGPT source:
def __truediv__(self, other):
    return self * other**-1

# Step 1: other_inv = other ** -1   (POW node, e=-1)
#   other_inv.data        = other.data ** -1 = 1/other
#   other_inv.children    = [other]
#   other_inv.local_grads = [e * other^(e-1)]
#                         = [-1 * other^(-2)]
#                         = [-1 / other^2]
#
# Step 2: result = self * other_inv   (MUL node)
#   result.data        = self.data * (1/other)
#   result.children    = [self, other_inv]
#   result.local_grads = [other_inv, self.data]
#                      = [1/other,  self.data]
```

```
# Worked example: a=6, b=3  ->  a/b
a, b = Value(6), Value(3)
result = a / b

# Forward:
#   b_inv.data  = 3**-1 = 0.333
#   result.data = 6 * 0.333 = 2.0
#
# Backward (seed=1 at result):
#   1. result (MUL):
#      a.grad     += b_inv * 1.0 = 0.333
#      b_inv.grad += a     * 1.0 = 6
#   2. b_inv (POW, e=-1):
#      b.grad += (-1/b^2) * b_inv.grad
#             = (-1/9) * 6 = -0.667
#
# a.grad=0.333, b.grad=-0.667  checkmark
# Verify: d/da(a/b)=1/b=0.333  checkmark
# Verify: d/db(a/b)=-a/b^2=-0.667  checkmark
```

```
# Chain rule traced through 2-node decomposition:
#
#   [b=3] ---> [POW e=-1, data=0.333] ---> [MUL, data=2]
#   [a=6] ----------------------------------------/
#
```

__RTRUEDIV__

__rtruediv__ — other / self = other * self**-1

Called when a scalar is on the LEFT side of division (e.g. 1.0 / tensor). Here 'self' is in the denominator. The gradient of self is NEGATIVE and depends on self's own value — different from __truediv__.

```
# MicroGPT source:
def __rtruediv__(self, other):
    return other * self**-1

# When called:  6 / tensor  -> tensor.__rtruediv__(6)
#
# Step 1: self_inv = self ** -1  (POW node, e=-1)
#   self_inv.data        = self.data**-1 = 1/self
#   self_inv.local_grads = [-1 / self^2]
#
# Step 2: result = Value(other) * self_inv  (MUL node)
#   result.data        = other * (1/self)
#   result.children    = [Value(other), self_inv]
#   result.local_grads = [self_inv=1/self, other]
#
# Backward for self:
#   self_inv.grad += other * upstream  (from MUL)
#   self.grad     += (-1/self^2) * self_inv.grad
#                  = (-1/self^2) * other * upstream
#                  = -other/self^2 * upstream
```

```
# Worked example: 6 / a  (scalar / Value)
a = Value(3)
result = 6 / a      # calls a.__rtruediv__(6)

# Forward:
#   a_inv.data  = 3**-1 = 0.333
#   result.data = 6 * 0.333 = 2.0
#
# Backward (seed=1 at result):
#   1. result (MUL):
#      Value(6).grad += a_inv * 1 = 0.333 (ignored)
#      a_inv.grad    += 6     * 1 = 6
#   2. a_inv (POW, e=-1):
#      a.grad += (-1/a^2) * a_inv.grad
#             = (-1/9)   * 6 = -0.667
#
# a.grad = -0.667
# Verify: d/da(6/a) = -6/a^2 = -6/9 = -0.667  checkmark
#
# Compare __truediv__:  (a/6) -> a.grad = 1/6 = +0.167
# Compare __rtruediv__: (6/a) -> a.grad = -6/a^2 (negative!)
```

__truediv__(a,b): a/b -> a.grad=+1/b, b.grad=-a/b² | __rtruediv__(a,b): b/a -> a.grad=-b/a² (a is in denominator, gets negative grad)

What We Covered

Understanding the Math Behind MicroGPT by Andrej Karpathy

01

Partial derivatives measure how each input independently affects output. Tensors extend scalars by applying rules elementwise at each index i.

02

Chain rule: dL/dx = dL/dy × dy/dx. For composed graphs, multiply local derivatives back through each node. Node reuse requires += accumulation.

03

DAG + topological sort guarantees correctness: every node's .grad is fully accumulated before it propagates. The backward() DFS ensures this.

04

All elementwise Jacobians are diagonal. The O(n) shortcut grad = upstream ⊗ f'(u) replaces an n×n matrix multiply in every practical framework.

05

Shape ops (reshape, broadcast, concat, transpose) reverse structurally. Min/max route gradient to the winner. Softmax is the one dense exception.

APPENDIX

Numerical Gradient Checking — Verify Your Backward Pass

Numerical gradient checking is the gold standard for verifying backprop implementations. Compare analytic gradients against finite difference approximations.

```
def numerical_gradient(f, x, eps=1e-5):
    """Finite difference gradient check."""
    grad = np.zeros_like(x)
    for i in range(x.size):
        x_plus  = x.copy(); x_plus.flat[i]  += eps
        x_minus = x.copy(); x_minus.flat[i] -= eps
        grad.flat[i] = (f(x_plus) - f(x_minus)) / (2 *
eps)
    return grad

# Usage:
analytic = backward_pass(x)
numeric  = numerical_gradient(f, x)

# Compare:
rel_error = np.abs(analytic - numeric) / (
    np.abs(analytic) + np.abs(numeric) + 1e-8)
print('Max relative error:', rel_error.max())
# Should be < 1e-5 for float64
```

```
# Common gotchas:
#
# 1. Forget += for node reuse:
#    a.grad = instead of a.grad +=
#    -> gradient overwritten, wrong!
#
# 2. Use input in backward, not output:
#    For mul: grad_A = upstream * B   (correct)
#             grad_A = upstream * A   (wrong!)
#
# 3. Missing chain rule factor:
#    For log: grad = 1 (wrong)
#             grad = upstream / A (correct)
#
# 4. Wrong sign for subtraction:
#    grad_B = upstream (wrong)
#    grad_B = -upstream (correct)
#
# 5. Broadcast shape mismatch:
#    After sum(), need .reshape(A.shape)
```

Rule of thumb: relative error < 1e-5 is acceptable, > 1e-3 is almost certainly wrong. Test each op in isolation.

REFERENCES

References

01

microgpt — Building a Small GPT from Scratch

Andrej Karpathy

Original microgpt implementation and blog post covering the full architecture, autograd engine, and training loop.

http://karpathy.github.io/2026/02/12/microgpt/

02

CIS 5690 — GPU Programming for Machine Learning

Course Reference

University of Pennsylvania

Graduate course covering GPU-accelerated deep learning, CUDA programming, and systems-level ML implementations including autograd.

University of Pennsylvania · CIS 5690